Software Engineering **TD 7**

Software Architecture - MVC **3RD YEAR - ENSIA**

OBJECTIVES

- Choose the appropriate Architectural Pattern.
- Apply the MVC pattern to separate data, interface, and logic.
- Organize system components using UML Package Diagrams.

TEAM EXERCISES

Task / Exercise C1: Architecture Pattern Matching

For each scenario below, suggest the most appropriate architectural style and justify your choice.

Scenario	Best Architecture	Justification
A Unix Shell Command: You are building a system that reads a text file, removes stop-words, sorts the remaining words alphabetically, and then counts the frequency of each word. The output of one step is the input of the next.		
A banking system where different teams need to independently develop and deploy the account, transaction, and loan modules		
A mobile coffee ordering app with a backend API		
A stock trading platform where price changes must trigger automatic buy/sell orders across thousands of users		
A university management system with separate authentication, student records, course management, and billing subsystems		
An image recognition system that processes images through detection → classification → labeling stages		

Task / Exercise 2: Package Diagram (System Organization)

Context:

You are the lead architect for the "Smart Home Automation System." The development team has created many classes, but the project structure is becoming messy. There is no clear organization, and files are scattered everywhere.

The List of Classes:

The team has identified the following major classes/modules in the system:

1. **LoginScreen**: Renders the input fields for username/password and displays authentication error messages to the user.
2. **ThermostatDriver**: Contains low-level code that reads raw voltage signals from the temperature sensor via the serial port (GPIO).
3. **AutomationController**: The "brain" of the system. It compares current sensor readings against user-defined rules to decide if the heater should turn On or Off.
4. **UserConfigRepository**: Handles the technical details of writing user settings (JSON/XML) to the hard drive and reading them back.
5. **PostgreSQLConnection**: Manages the database connection pool, authentication strings, and raw SQL transaction commits.
6. **MotionSensorDriver**: Listens for hardware interrupts on the device pins to detect physical movement and converts them into software events.
7. **DashboardView**: Generates the graphical charts showing historical energy usage and the current status of the house.
8. **SecurityManager**: Contains the logic to determine if an event is a security threat (e.g., "If Window Open AND Alarm Armed THEN Trigger Breach").
9. **EmailService**: A wrapper for the SMTP protocol that connects to an external Gmail server to send alerts over the internet.

Instructions:

Draw a **UML Package Diagram** to organize these classes into logical groups (namespaces).

1. **Create Packages:** Group the classes above into **4 logical packages.** (Suggested names: *Presentation*, *Logic/Core*, *Hardware/Devices*, *Data/Infrastructure*).
2. **Assign Content:** Write the class names inside their respective package folders.
3. **Draw Dependencies:** Draw dashed arrows (**<<import>>** or **<<access>>**) between the packages to show which package needs to use the other.
    - *Constraint:* Ensure there are no circular dependencies (e.g., Package A depends on B, and B depends on A).
    - *Hint:* Does the *Database* need to know about the *UI*? Does the *UI* need to know about the *Hardware Drivers* directly, or should it go through the *Logic*?

Discussion Question:
Why is it considered bad practice if your **Data/Infrastructure** package has a dependency pointing upwards to the **Presentation** package?

Task / Exercise 3: The MVC Pattern

Context:
Your team is building the "Student Gradebook System". A teacher uses this web application to enter marks for students. You have chosen the MVC (Model-View-Controller) architectural pattern to ensure the code is organized and maintainable.

Part 1: Classifying Components

Below is a list of system components and responsibilities. For each item, indicate whether it belongs to the Model, the View, or the Controller.

- **StudentGrade Class**: A class that holds data (Student ID, Course ID, Mark) and contains logic to calculate if the grade is a "Pass" or "Fail" (mark $\geqslant$ 10).
- **GradeEntryForm.html**: An HTML/CSS page showing input fields for the student ID and the mark.
- **SubmitGradeHandler**: A function/class that receives the "Click" event when the teacher hits the "Save" button.

- **Grades Database Table Schema**: The SQL table structure that defines how raw grades are stored (columns: student_id, course_id, mark, date).
- **ErrorPopup**: A window that displays the message "Invalid Input: Grade cannot be negative" to the user.
- **NotificationService**: A background process that automatically sends an email notification when a grade is updated in the database (triggered by changes to the Model's state).

Part 2: Tracing the Flow

The teacher attempts to enter a grade of "25/20" (an invalid grade, since maximum is 20) for a student and clicks "Save".

Instructions:

1- Reorder the following steps (label them 1 to 5) to show the correct flow of events in a strict MVC architecture.

2- Briefly explain why this order follows MVC principles.

(___) The View displays an error message: "Grade cannot exceed 20."

(___) The Controller receives the input "25" from the form.

(___) The Model validates the data and returns an error/exception because 25 > 20.

(___) The Controller interprets the error from the Model and decides which screen to show next.

(___) The User interacts with the View by typing "25" and clicking "Save".

Part C: The "Fat Controller" Problem

A junior developer submits the following pseudocode for the Controller. This code violates the MVC pattern.

```
// Controller Code
Function OnSubmitButton_Click(studentID, mark) {
    // 1. Validation Logic
    IF mark < 0 OR mark > 20 THEN
        Return "Error: Mark invalid" (HTML string)
    END IF

    // 2. Database Logic
    DatabaseConnection db = new DatabaseConnection("sql_server_url");
    db.Execute("UPDATE Grades SET value = " + mark + " WHERE id = " + studentID);

    // 3. Calculation Logic
```

```
    String status = "";
    IF mark >= 10 THEN
        status = "Pass";
    ELSE
        status = "Fail";
    END IF

    // 4. UI Generation
    Return "<html><body><h1>Saved: " + status + "</h1></body></html>";
}
```

Questions:

1. Identify three distinct violations of MVC in this code snippet. For each violation:
   - State what the Controller is doing that it shouldn't
   - Explain which component (Model or View) should handle it instead
2. If the university decides to switch from a Web App to a Mobile App, which part(s) of the code above would become useless and force a complete rewrite? Why does strict MVC help prevent this problem?
3. Refactor the validation logic. Rewrite the code below to show proper MVC structure (use a pseudocode). You should create:
   - A Model class/function that handles validation
   - A Controller function that uses the Model
   - Show how errors would be passed to the View (you don't need to write the View code)

Part D:

The system needs a new feature: automatically calculate the class average whenever any grade is updated, and display it on a dashboard.

- Describe which components (Model/View/Controller) would be involved and what each would do. Then explain where this calculation logic should live and why.